

Big Data Analytics Lab Manual

Step-by-step: Commands, then Program, then Run Commands and Output (Ubuntu, Labs 1 - 10)

How every lab is arranged

Each lab follows the same order, so you can just go from top to bottom:

Colour	What it means	What you do
Blue - COMMANDS	Commands for the terminal (setup, open editor, compile, run).	Copy and paste into the terminal and press Enter.
Green - PROGRAM	The program code. It always comes right after the nano filename command.	Paste it inside the nano editor, then save: Ctrl+O, Enter, Ctrl+X.
Grey - OUTPUT	What you should see after running.	Compare with your screen.

Where do I write the programs? (nano editor in the terminal)

No extra software is needed. The programs are written in the terminal with **nano**, a text editor that is already installed on Ubuntu:

1. Open the terminal: `Ctrl + Alt + T`
2. Type: `nano program_name.py` (then Enter)
3. nano opens a blank page. Paste the code: `Ctrl + Shift + V` (or right-click -> Paste)
4. Save: `Ctrl + O`, then press Enter
5. Exit nano: `Ctrl + X`
6. Check that the file is saved: `cat program_name.py`

Important: (1) The file name must be exactly as written, including capital letters. For Java, WordCount.java must contain the class WordCount. (2) Copy a command block **completely**. Blocks that start with `cat > file << 'EOF'` end with a line that says only EOF. Paste all of it, including that last line. (3) Do not use sudo in front of hdfs or hadoop commands. (4) Lab order: do Lab 1 first, because Labs 3 to 7 need HDFS. (5) Work as the **hadoop** user: in every new terminal first run `sudo su - hadoop` (the user is created in Lab 1).

Assumptions

Ubuntu 22.04 or 24.04 (64-bit), internet connection, a user with sudo rights, Java 8 and Hadoop 3.3.6 (HDFS + YARN, started with start-all.sh, as in your ma'am's steps). I corrected the commands from the manual but could not run them on a live Ubuntu machine, so if any command shows an error, see the troubleshooting table at the end.

Lab 1: Installation, Configuration and Verification

Objective: Install and verify NumPy, Pandas, Matplotlib, Seaborn and single-node Hadoop (HDFS + YARN).

If Hadoop is already installed on your computer (from your ma'am's reference steps), do not install it again. Just run `sudo su - hadoop`, check `start-all.sh` and `jps`, and continue with the Python steps and the verification program at the end of this lab.

STEP 1 | COMMANDS - create the hadoop user and switch to it (all later steps are done as this user)

```
sudo adduser hadoop
sudo usermod -aG sudo hadoop
sudo su - hadoop
```

adduser asks for a password (remember it) and some details (just press Enter). If it says the user already exists, that is fine: continue with the next command. The prompt must now show **hadoop@....**

STEP 2 | COMMANDS - install Python and libraries (virtual environment)

```
sudo apt update
sudo apt install python3 python3-pip python3-venv -y
python3 -m venv ~/bda-env
source ~/bda-env/bin/activate
pip install numpy pandas matplotlib seaborn
```

In every new terminal, first run `source ~/bda-env/bin/activate` before Python labs (1, 2, 8, 9, 10). The prompt then shows (bda-env).

STEP 3 | COMMANDS - install Java 8 and the SSH server

```
sudo apt install openjdk-8-jdk openssh-server -y
sudo systemctl enable --now ssh
java -version
javac -version
```

STEP 4 | COMMANDS - passwordless SSH to localhost (needed by start-dfs.sh)

```
ssh-keygen -t rsa -P "" -f ~/.ssh/id_rsa
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
ssh localhost
# type "yes" the first time, then leave with:
exit
```

STEP 5 | COMMANDS - download and install Hadoop 3.3.6

```
cd ~
wget https://archive.apache.org/dist/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
tar -xzf hadoop-3.3.6.tar.gz
sudo mv hadoop-3.3.6 /usr/local/hadoop
sudo chown -R $USER:$USER /usr/local/hadoop
```

STEP 6 | COMMANDS - set environment variables

```
cat << 'EOF' >> ~/.bashrc
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
export HADOOP_HOME=/usr/local/hadoop
export HADOOP_MAPRED_HOME=/usr/local/hadoop
export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
EOF
source ~/.bashrc
hadoop version
```

On an ARM computer use `java-8-openjdk-arm64` instead of `amd64`. To check: `ls /usr/lib/jvm`.

STEP 7 | COMMANDS - configure single-node HDFS

```
echo "export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64" >> $HADOOP_HOME/etc/hadoop/hadoop-env.sh

mkdir -p ~/hadoop_data/namenode ~/hadoop_data/datanode

cat > $HADOOP_HOME/etc/hadoop/core-site.xml << 'EOF'
<?xml version="1.0"?>
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
EOF

# Note: no quotes around EOF in the next block, so $HOME is filled in automatically
cat > $HADOOP_HOME/etc/hadoop/hdfs-site.xml << EOF
<?xml version="1.0"?>
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file://$HOME/hadoop_data/namenode</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file://$HOME/hadoop_data/datanode</value>
  </property>
</configuration>
EOF
```

STEP 8 | COMMANDS - configure YARN (needed for MapReduce jobs and Pig MapReduce mode)

```
cat > $HADOOP_HOME/etc/hadoop/mapred-site.xml << 'EOF'
<?xml version="1.0"?>
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
  <property>
    <name>mapreduce.application.classpath</name>
    <value>$HADOOP_MAPRED_HOME/share/hadoop/mapreduce/*:$HADOOP_MAPRED_HOME/share/hadoop/mapreduce/lib/*</value>
  </property>
</configuration>
EOF

WL="JAVA_HOME,HADOOP_COMMON_HOME,HADOOP_HDFS_HOME,HADOOP_CONF_DIR,"
WL="$WL}CLASSPATH_PREPEND_DISTCACHE,HADOOP_YARN_HOME,HADOOP_HOME,PATH,LANG,TZ,HADOOP_MAPRED_HOME"

# Note: no quotes around EOF in the next block, so $WL is filled in automatically
cat > $HADOOP_HOME/etc/hadoop/yarn-site.xml << EOF
<?xml version="1.0"?>
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
  <property>
    <name>yarn.nodemanager.env-whitelist</name>
    <value>$WL</value>
  </property>
  <property>
    <name>yarn.nodemanager.vmem-check-enabled</name>
    <value>false</value>
  </property>
</configuration>
EOF
```

STEP 9 | COMMANDS - format the NameNode (ONLY ONCE), then start HDFS and YARN

```
hdfs namenode -format
start-all.sh
jps
```

jps must show **NameNode**, **DataNode**, **SecondaryNameNode**, **ResourceManager**, **NodeManager** and Jps. Web pages: <http://localhost:9870> (HDFS) and <http://localhost:8088> (YARN). Do not run `hdfs namenode -format` again later. If `start-all.sh` is not found use `start-dfs.sh` and `start-yarn.sh`. Stop everything with `stop-all.sh`.

STEP 10 | COMMANDS - make a folder for this lab

```
mkdir -p ~/bda_labs/lab1
cd ~/bda_labs/lab1
```

STEP 11 | COMMAND - open the editor for lab1_setup.py

```
nano lab1_setup.py
```

STEP 12 | PROGRAM - copy this code and paste it inside nano (lab1_setup.py)

```
import numpy as np
import pandas as pd

# NumPy Verification
arr = np.array([10, 20, 30, 40, 50])
print(f"NumPy Mean: {np.mean(arr)}")

# Pandas Verification
df = pd.DataFrame({
    'ID': [101, 102, 103],
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Score': [85, 92, 78]
})
print("\nPandas DataFrame:")
print(df)
```

Now save and exit nano: press **Ctrl+O**, then **Enter**, then **Ctrl+X**.

STEP 13 | RUN COMMANDS - run the program

```
source ~/bda-env/bin/activate
python3 lab1_setup.py
```

EXPECTED OUTPUT

```
NumPy Mean: 30.0
```

```
Pandas DataFrame:
   ID  Name  Score
0  101  Alice    85
1  102   Bob    92
2  103 Charlie    78
```

Lab 2: Data Visualization in Python

Objective: Line plot, bar chart, histogram and scatter plot using Matplotlib and Seaborn.

Needs: only Python (Hadoop is not needed for this lab).

STEP 1 | COMMANDS - go to a lab folder and activate Python

```
source ~/bda-env/bin/activate
mkdir -p ~/bda_labs/lab2
cd ~/bda_labs/lab2
```

STEP 2 | COMMAND - open the editor for lab2_visualization.py

```
nano lab2_visualization.py
```

STEP 3 | PROGRAM - copy this code and paste it inside nano (lab2_visualization.py)

```
import matplotlib
matplotlib.use("Agg")          # saves the image without needing a display window
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

sns.set_theme()

# Generate dummy dataset
np.random.seed(42)
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
sales = [12000, 15000, 18000, 14000, 22000, 25000]
df = pd.DataFrame({'Month': months, 'Sales': sales})

# Create a multi-panel plot layout
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

# 1. Line Plot
axes[0, 0].plot(df['Month'], df['Sales'], marker='o', color='b', linewidth=2)
axes[0, 0].set_title('Monthly Sales Trend (Line Plot)')

# 2. Bar Chart
axes[0, 1].bar(df['Month'], df['Sales'], color='skyblue')
axes[0, 1].set_title('Monthly Revenue Comparison (Bar Chart)')

# 3. Histogram
data = np.random.normal(loc=100, scale=15, size=500)
axes[1, 0].hist(data, bins=20, color='lightgreen', edgecolor='black')
axes[1, 0].set_title('Distribution Analysis (Histogram)')

# 4. Scatter Plot
x = np.random.rand(50) * 100
y = x * 2 + np.random.randn(50) * 10
axes[1, 1].scatter(x, y, color='purple', alpha=0.7)
axes[1, 1].set_title('Scatter Relationship Plot')

plt.tight_layout()
plt.savefig('visualization_output.png')
print("Plot successfully saved as 'visualization_output.png'")
```

Now save and exit nano: press **Ctrl+O**, then **Enter**, then **Ctrl+X**.

STEP 4 | RUN COMMANDS - run the program and copy the picture to a place where you can open it

```
python3 lab2_visualization.py
cp visualization_output.png /tmp/visualization_output.png
chmod 644 /tmp/visualization_output.png
```

Open the picture from the file manager at **/tmp/visualization_output.png** (or with `xdg-open /tmp/visualization_output.png` if you have a normal desktop session). The image is also saved in your lab folder.

EXPECTED OUTPUT

```
Plot successfully saved as 'visualization_output.png'
```

Lab 3: Basic HDFS Commands and File Operations

Objective: Read, write, copy and manage files and directories in HDFS. This lab has no program, only commands.

Needs: Hadoop running.

STEP 1 | COMMANDS - start Hadoop (skip if jps already shows NameNode and DataNode)

```
start-all.sh
jps
```

STEP 2 | COMMANDS - create a folder in HDFS and a local sample file

```
mkdir -p ~/bda_labs/lab3
cd ~/bda_labs/lab3
hdfs dfs -mkdir -p /user/hadoop/lab_data
echo "Hadoop HDFS File System Demonstration File" > sample_local.txt
```

STEP 3 | COMMANDS - write (upload), list and read the file

```
hdfs dfs -put sample_local.txt /user/hadoop/lab_data/
hdfs dfs -ls /user/hadoop/lab_data/
hdfs dfs -cat /user/hadoop/lab_data/sample_local.txt
```

STEP 4 | COMMANDS - copy inside HDFS and download back (each command is ONE line)

```
hdfs dfs -cp /user/hadoop/lab_data/sample_local.txt /user/hadoop/lab_data/backup.txt
hdfs dfs -get /user/hadoop/lab_data/backup.txt ./downloaded_backup.txt
cat downloaded_backup.txt
```

STEP 5 | COMMANDS - extra commands (optional)

```
hdfs dfs -ls /user/hadoop/lab_data/
hdfs dfs -du -h /user/hadoop/lab_data/
hdfs dfs -mv /user/hadoop/lab_data/backup.txt /user/hadoop/lab_data/backup2.txt
hdfs dfs -rm /user/hadoop/lab_data/backup2.txt
```

EXPECTED OUTPUT

```
$ hdfs dfs -ls /user/hadoop/lab_data/
Found 1 items
-rw-r--r--  1 <your-username> supergroup    43 <date time> /user/hadoop/lab_data/sample_local.txt

$ hdfs dfs -cat /user/hadoop/lab_data/sample_local.txt
Hadoop HDFS File System Demonstration File
```

The owner name in the listing is your Linux username and the date is today's date. That is normal.

Lab 4: MapReduce Applications

Lab 4a: Word Count

Needs: Hadoop and YARN running. The job appears at <http://localhost:8088>.

STEP 1 | COMMANDS - start Hadoop (skip if jps already shows ResourceManager and NodeManager)

```
start-all.sh  
jps
```

STEP 2 | COMMANDS - make a lab folder

```
mkdir -p ~/bda_labs/lab4a  
cd ~/bda_labs/lab4a
```

STEP 3 | COMMAND - open the editor for WordCount.java

```
nano WordCount.java
```

STEP 4 | PROGRAM - copy this code and paste it inside nano (WordCount.java)

```
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    public static class TokenizerMapper
        extends Mapper<Object, Text, Text, IntWritable> {
        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {
            StringTokenizer itr = new StringTokenizer(value.toString());
            while (itr.hasMoreTokens()) {
                word.set(itr.nextToken().toLowerCase().replaceAll("[^a-zA-Z]", ""));
                if (word.getLength() > 0) {
                    context.write(word, one);
                }
            }
        }
    }

    public static class IntSumReducer
        extends Reducer<Text, IntWritable, Text, IntWritable> {
        private IntWritable result = new IntWritable();

        public void reduce(Text key, Iterable<IntWritable> values, Context context)
            throws IOException, InterruptedException {
            int sum = 0;
            for (IntWritable val : values) {
                sum += val.get();
            }
            result.set(sum);
            context.write(key, result);
        }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "word count");
        job.setJarByClass(WordCount.class);
        job.setMapperClass(TokenizerMapper.class);
        job.setCombinerClass(IntSumReducer.class);
        job.setReducerClass(IntSumReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Now save and exit nano: press **Ctrl+O**, then **Enter**, then **Ctrl+X**.

STEP 5 | RUN COMMANDS - compile, make the jar, prepare input, run the job, see the result

```
export HADOOP_CLASSPATH=$(hadoop classpath)
javac -classpath $HADOOP_CLASSPATH -d . WordCount.java
jar cvf wordcount.jar WordCount*.class

echo "hadoop is big data framework. mapreduce processes big data." > input.txt
hdfs dfs -mkdir -p /wc/input
hdfs dfs -put -f input.txt /wc/input/

# the output folder must NOT exist, so remove it before every run
hdfs dfs -rm -r -f /wc/output

hadoop jar wordcount.jar WordCount /wc/input /wc/output
hdfs dfs -cat /wc/output/part-r-00000
```

EXPECTED OUTPUT

```
big      2
data     2
framework 1
hadoop   1
is        1
mapreduce 1
processes 1
```

In the real output the word and the number are separated by a TAB.

Lab 4b: Weather Dataset - Maximum Temperature

Three Java files are needed. Do them one after the other. Input lines look like *year temperature*.

STEP 6 | COMMANDS - make a lab folder

```
mkdir -p ~/bda_labs/lab4b
cd ~/bda_labs/lab4b
```

STEP 7 | COMMAND - open the editor for WeatherMapper.java

```
nano WeatherMapper.java
```

STEP 8 | PROGRAM - copy this code and paste it inside nano (WeatherMapper.java)

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class WeatherMapper extends Mapper<LongWritable, Text, Text, IntWritable> {
    @Override
    public void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {
        String[] parts = value.toString().trim().split("\\s+");
        if (parts.length >= 2) {
            try {
                String year = parts[0];
                int temp = Integer.parseInt(parts[1]);
                context.write(new Text(year), new IntWritable(temp));
            } catch (NumberFormatException e) {
                // skip malformed line
            }
        }
    }
}
```

Now save and exit nano: press **Ctrl+O**, then **Enter**, then **Ctrl+X**.

STEP 9 | COMMAND - open the editor for WeatherReducer.java

```
nano WeatherReducer.java
```

STEP 10 | PROGRAM - copy this code and paste it inside nano (WeatherReducer.java)

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class WeatherReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    @Override
    public void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {
        int maxTemp = Integer.MIN_VALUE;
        for (IntWritable val : values) {
            maxTemp = Math.max(maxTemp, val.get());
        }
        context.write(key, new IntWritable(maxTemp));
    }
}
```

Now save and exit nano: press **Ctrl+O**, then **Enter**, then **Ctrl+X**.

STEP 11 | COMMAND - open the editor for WeatherDriver.java

```
nano WeatherDriver.java
```

STEP 12 | PROGRAM - copy this code and paste it inside nano (WeatherDriver.java)

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WeatherDriver {
    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.err.println("Usage: WeatherDriver <input path> <output path>");
            System.exit(-1);
        }
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "max temperature");
        job.setJarByClass(WeatherDriver.class);
        job.setMapperClass(WeatherMapper.class);
        job.setCombinerClass(WeatherReducer.class);
        job.setReducerClass(WeatherReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Now save and exit nano: press **Ctrl+O**, then **Enter**, then **Ctrl+X**.

STEP 13 | RUN COMMANDS - compile, make the jar, create sample data, run, see the result

```
export HADOOP_CLASSPATH=$(hadoop classpath)
javac -classpath $HADOOP_CLASSPATH -d . WeatherMapper.java WeatherReducer.java WeatherDriver.java
jar cvf weather.jar Weather*.class

printf "2023 0035\n2023 0042\n2024 0038\n2024 0045\n" > sample_weather.txt

hdfs dfs -mkdir -p /weather/input
hdfs dfs -put -f sample_weather.txt /weather/input/
hdfs dfs -rm -r -f /weather/output
hadoop jar weather.jar WeatherDriver /weather/input /weather/output
hdfs dfs -cat /weather/output/part-r-00000
```

EXPECTED OUTPUT

```
2023    42
2024    45
```

Lab 4c: Real-World Case Study Overview (theory)

Industry	MapReduce job profile	Input data	Output / business value
E-Commerce	Recommendation engines and association rules	User clickstream logs	Product co-occurrence counts
Telecommunications	Call Detail Record (CDR) aggregation	Network logs	Peak usage and network load maps
Finance	Batch fraud analysis	Nightly transaction batches	Anomaly detection flags

Lab 5: Apache Pig - User Defined Function (UDF)

Objective: Write an Eval UDF in Java to clean and filter records in a Pig script. This lab follows your ma'am's steps: input and output are in **HDFS** and Pig runs in **MapReduce mode**.

STEP 1 | COMMANDS - switch to the Hadoop user (skip this if you installed Hadoop under your own user name)

```
sudo su - hadoop
```

STEP 2 | COMMANDS - check Java (Java 8 must be available)

```
java -version  
javac -version
```

STEP 3 | COMMANDS - start Hadoop and YARN

```
start-all.sh  
jps
```

jps should show **NameNode**, **DataNode**, **SecondaryNameNode**, **ResourceManager**, **NodeManager** and **Jps**. If **start-all.sh** is not available, use: **start-dfs.sh** and then **start-yarn.sh**, then **jps**.

If **ResourceManager** or **NodeManager** is missing in **jps**, YARN is not configured: do the "configure YARN" step of Lab 1, then run **stop-all.sh** and **start-all.sh** again.

STEP 4 | COMMANDS - install Apache Pig 0.17.0

```
cd ~  
wget https://archive.apache.org/dist/pig/pig-0.17.0/pig-0.17.0.tar.gz  
tar -xzf pig-0.17.0.tar.gz  
sudo mv pig-0.17.0 /usr/local/pig  
sudo chown -R $USER:$USER /usr/local/pig
```

STEP 5 | COMMANDS - open the .bashrc file to add the Pig variables

```
nano ~/.bashrc
```

STEP 6 | ADD LINES - go to the very end of the file and paste these lines

```
export PIG_HOME=/usr/local/pig  
export PATH=$PATH:$PIG_HOME/bin  
export PIG_CLASSPATH=$HADOOP_HOME/etc/hadoop  
export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop
```

Now save and exit nano: press **Ctrl+O**, then **Enter**, then **Ctrl+X**.

STEP 7 | COMMANDS - load the new settings and check Pig

```
source ~/.bashrc  
pig -version  
echo $PIG_HOME  
echo $HADOOP_CONF_DIR
```

If **pig: command not found** appears, run **source ~/.bashrc** again and check that **/usr/local/pig/bin/pig** exists. (**HADOOP_HOME** must already be set from Lab 1.)

STEP 8 | COMMANDS - create the working directory for Lab 5

```
mkdir -p ~/lab5_pig_udf/com/lab/pig  
cd ~/lab5_pig_udf  
pwd
```

STEP 9 | COMMAND - open the editor for com/lab/pig/CleanDataUDF.java

```
nano com/lab/pig/CleanDataUDF.java
```

STEP 10 | PROGRAM - copy this code and paste it inside nano (com/lab/pig/CleanDataUDF.java)

```
package com.lab.pig;

import java.io.IOException;
import org.apache.pig.EvalFunc;
import org.apache.pig.data.Tuple;

public class CleanDataUDF extends EvalFunc<String> {
    @Override
    public String exec(Tuple input) throws IOException {
        if (input == null || input.size() == 0 || input.get(0) == null) {
            return null;
        }
        String str = (String) input.get(0);
        // Remove special symbols, trim spaces and convert to uppercase
        String cleaned = str
            .replaceAll("[^a-zA-Z0-9 ]", "")
            .trim()
            .toUpperCase();
        return cleaned.isEmpty() ? null : cleaned;
    }
}
```

Now save and exit nano: press **Ctrl+O**, then **Enter**, then **Ctrl+X**.

Careful: in the text copy of the ma'am's file the comment line and the next line were joined (...uppercaseString cleaned = str). That would comment out the code and the compile would fail. In the program above the comment is on its own line. Make sure it stays on its own line in nano.

STEP 11 | COMMANDS - compile the UDF and check the class file

```
cd ~/lab5_pig_udf
javac -cp "$PIG_HOME/pig-0.17.0-core-h2.jar:${hadoop classpath}" -d . com/lab/pig/CleanDataUDF.java
find com -name "CleanDataUDF.class" -print
```

Expected path: com/lab/pig/CleanDataUDF.class

STEP 12 | COMMANDS - create the JAR file

```
cd ~/lab5_pig_udf
jar -cvf CleanDataUDF.jar com/lab/pig/CleanDataUDF.class
ls -lh CleanDataUDF.jar
```

STEP 13 | COMMANDS - go to the folder and open the sample input file

```
cd ~/lab5_pig_udf
nano raw_logs.csv
```

STEP 14 | DATA - paste these 3 lines inside nano (raw_logs.csv)

```
1,Valid user session active!
2,Purchase completed successfully.
3,@@@
```

Now save and exit nano: press **Ctrl+O**, then **Enter**, then **Ctrl+X**.

STEP 15 | COMMANDS - check the file and upload it to HDFS

```
cat raw_logs.csv
hdfs dfs -mkdir -p /user/hadoop/pig_input
hdfs dfs -rm -f /user/hadoop/pig_input/raw_logs.csv
hdfs dfs -put ~/lab5_pig_udf/raw_logs.csv /user/hadoop/pig_input/
hdfs dfs -ls /user/hadoop/pig_input
hdfs dfs -cat /user/hadoop/pig_input/raw_logs.csv
```

If your Hadoop username is different, you can still use /user/hadoop as written (as in Lab 3). If your ma'am's setup uses another HDFS user folder, replace /user/hadoop in all commands and in the Pig script.

STEP 16 | COMMANDS - go to the folder and open the Pig script

```
cd ~/lab5_pig_udf
nano filter_script.pig
```

STEP 17 | PROGRAM - paste this Pig script inside nano (filter_script.pig)

```
-- Register custom JAR file
REGISTER 'CleanDataUDF.jar';

-- Load raw data from HDFS
raw_data = LOAD '/user/hadoop/pig_input/raw_logs.csv'
            USING PigStorage(',')
            AS (id:int, message:chararray);

-- Clean data using the Java User Defined Function
cleaned_tuples = FOREACH raw_data
                 GENERATE id,
                 com.lab.pig.CleanDataUDF(message) AS clean_msg;

-- Filter out null/unwanted records
filtered_data = FILTER cleaned_tuples
               BY clean_msg IS NOT NULL;

-- Store the final result in HDFS
STORE filtered_data
INTO '/user/hadoop/pig_output'
USING PigStorage(',');
```

Now save and exit nano: press **Ctrl+O**, then **Enter**, then **Ctrl+X**.

STEP 18 | COMMANDS - remove any old output folder (Pig will not overwrite it)

```
hdfs dfs -rm -r -f /user/hadoop/pig_output
```

STEP 19 | RUN COMMANDS - run Pig in MapReduce mode (from the lab folder)

```
cd ~/lab5_pig_udf
pig -x mapreduce filter_script.pig
```

Pig compiles the script, submits a job to Hadoop/YARN and ends with a success message. Job IDs and times are different on every system.

STEP 20 | RUN COMMANDS - list and display the output

```
hdfs dfs -ls /user/hadoop/pig_output
hdfs dfs -cat /user/hadoop/pig_output/part-m-00000
```

EXPECTED OUTPUT

```
1,VALID USER SESSION ACTIVE
2,PURCHASE COMPLETED SUCCESSFULLY
```

Record 3 (@@@) contains only characters that the UDF removes, so the cleaned string is empty, the UDF returns null and FILTER removes that record.

STEP 21 | COMMANDS - ONLY IF MapReduce mode fails with a Hadoop-version error: run Pig in local mode instead

```
cd ~/lab5_pig_udf
sed "s#/user/hadoop/pig_input/##; s#/user/hadoop/pig_output#pig_output#" filter_script.pig > filter_local.pig
rm -rf pig_output
pig -x local filter_local.pig
cat pig_output/part-m-00000
```

Pig 0.17.0 was built for Hadoop 2.x. It often works on Hadoop 3.x in MapReduce mode (as in your ma'am's steps), but if it does not, the local-mode block above gives the same output without HDFS.

Lab 6: NoSQL Operations using MongoDB

Objective: CRUD operations and embedded arrays in the MongoDB shell. This lab has no file: the program is pasted directly at the **mongosh** prompt.

Needs: only MongoDB (Hadoop is not needed).

STEP 1 | COMMANDS - install MongoDB 8.0 Community (Ubuntu 22.04 / 24.04), start it and open the shell

```
sudo apt-get install -y gnupg curl
curl -fsSL https://www.mongodb.org/static/pgp/server-8.0.asc | \
  sudo gpg -o /usr/share/keyrings/mongodb-server-8.0.gpg --dearmor
echo "deb [ arch=amd64,arm64 signed-by=/usr/share/keyrings/mongodb-server-8.0.gpg ] \
https://repo.mongodb.org/apt/ubuntu $(lsb_release -cs)/mongodb-org/8.0 multiverse" \
  | sudo tee /etc/apt/sources.list.d/mongodb-org-8.0.list
sudo apt-get update
sudo apt-get install -y mongodb-org
sudo systemctl start mongod
sudo systemctl status mongod
mongosh
```

STEP 2 | PROGRAM - paste this at the mongosh prompt (the line starting with test>)

```
use store_db

// start clean so re-running does not give a duplicate _id error
db.products.drop()

// 1. CREATE (insert documents with embedded arrays)
db.products.insertMany([
  { _id: 1, item: "Laptop", price: 1200,
    tags: ["electronics", "computers", "tech"], ratings: [5, 4, 5] },
  { _id: 2, item: "Mouse", price: 25,
    tags: ["electronics", "accessories"], ratings: [3, 4] }
])

// 2. READ (find with array filter)
db.products.find({ tags: "tech" })

// 3. UPDATE (change price and append a new tag)
db.products.updateOne(
  { _id: 1 },
  { $set: { price: 1150 }, $push: { tags: "sale" } }
)
db.products.find({ _id: 1 })

// 4. ARRAY AGGREGATION: unwind tags and count them
db.products.aggregate([
  { $unwind: "$tags" },
  { $group: { _id: "$tags", count: { $sum: 1 } } }
])

// 5. DELETE
db.products.deleteOne({ _id: 2 })
db.products.find()
```

To leave mongosh: type **exit** and press Enter.

EXPECTED OUTPUT

```
// after step 3: db.products.find({ _id: 1 })
{
  _id: 1,
  item: 'Laptop',
  price: 1150,
  tags: [ 'electronics', 'computers', 'tech', 'sale' ],
  ratings: [ 5, 4, 5 ]
}

// after step 4 (order of rows may differ)
{ _id: 'electronics', count: 2 }
{ _id: 'computers', count: 1 }
{ _id: 'tech', count: 1 }
{ _id: 'sale', count: 1 }
{ _id: 'accessories', count: 1 }
```

Lab 7: Hive and User Defined Functions

Lab 7a: HiveQL DDL/DML operations

Needs: Hadoop and YARN running.

STEP 1 | COMMANDS - start Hadoop (skip if jps already shows ResourceManager and NodeManager)

```
start-all.sh
jps
```

STEP 2 | COMMANDS - install Hive 3.1.3 and prepare it

```
cd ~
wget https://archive.apache.org/dist/hive/hive-3.1.3/apache-hive-3.1.3-bin.tar.gz
tar -xzf apache-hive-3.1.3-bin.tar.gz
sudo mv apache-hive-3.1.3-bin /usr/local/hive
sudo chown -R $USER:$USER /usr/local/hive

cat << 'EOF' >> ~/.bashrc
export HIVE_HOME=/usr/local/hive
export PATH=$PATH:$HIVE_HOME/bin
EOF
source ~/.bashrc

# Fix Guava version clash between Hive 3.1.3 and Hadoop 3.3.6
rm $HIVE_HOME/lib/guava-19.0.jar
cp $HADOOP_HOME/share/hadoop/common/lib/guava-27.0-jre.jar $HIVE_HOME/lib/

# Folders Hive needs in HDFS
hdfs dfs -mkdir -p /tmp /user/hive/warehouse
hdfs dfs -chmod g+w /tmp /user/hive/warehouse

# Lab folder + metastore (do this ONCE)
mkdir -p ~/hive_lab
cd ~/hive_lab
schematool -dbType derby -initSchema
```

Always run Hive commands from ~/hive_lab (the Derby metastore folder is created in the current folder). Only one Hive session can use it at a time. SLF4J "multiple bindings" warnings can be ignored.

STEP 3 | COMMANDS - create the sample data file

```
cd ~/hive_lab
cat > emp_data.csv << 'EOF'
1,Alice,IT,60000
2,Bob,HR,45000
3,Carol,IT,70000
4,Dave,HR,52000
5,Eve,Finance,55000
EOF
```

STEP 4 | COMMAND - open the editor for lab7a.hql

```
nano lab7a.hql
```

STEP 5 | PROGRAM - copy this code and paste it inside nano (lab7a.hql)

```
-- Create database and table
CREATE DATABASE IF NOT EXISTS lab_hive;
USE lab_hive;

CREATE TABLE IF NOT EXISTS employee (
  emp_id INT,
  name   STRING,
  dept   STRING,
  salary DOUBLE
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;

-- Load data (${env:HOME} is replaced by your home folder automatically)
LOAD DATA LOCAL INPATH '${env:HOME}/hive_lab/emp_data.csv' INTO TABLE employee;

-- Aggregation query (use the aggregate itself in HAVING, not the alias)
SELECT dept, COUNT(*) AS total_emp, AVG(salary) AS avg_sal
FROM employee
GROUP BY dept
HAVING AVG(salary) > 50000;
```

Now save and exit nano: press **Ctrl+O**, then **Enter**, then **Ctrl+X**.

STEP 6 | RUN COMMANDS - run the HiveQL file

```
cd ~/hive_lab
hive -f lab7a.hql
```

EXPECTED OUTPUT

Finance	1	55000.0
IT	2	65000.0

HR is left out because its average salary (48500) is below 50000. Row order may vary.

Lab 7b: Hive Java UDF

STEP 7 | COMMAND - open the editor for MaskSalaryUDF.java

```
nano MaskSalaryUDF.java
```

STEP 8 | PROGRAM - copy this code and paste it inside nano (MaskSalaryUDF.java)

```
package com.lab.hive;

import org.apache.hadoop.hive.ql.exec.UDF;
import org.apache.hadoop.io.Text;

public class MaskSalaryUDF extends UDF {
  public Text evaluate(Text input) {
    if (input == null) {
      return null;
    }
    return new Text("XXXX-CONFIDENTIAL");
  }
}
```

Now save and exit nano: press **Ctrl+O**, then **Enter**, then **Ctrl+X**.

STEP 9 | COMMANDS - compile and make the jar (run from ~/hive_lab)

```
cd ~/hive_lab
javac -cp "$(hadoop classpath):$HIVE_HOME/lib/*" -d . MaskSalaryUDF.java
jar cvf MaskSalaryUDF.jar com
```

STEP 10 | COMMAND - open the editor for lab7b.hql

```
nano lab7b.hql
```

STEP 11 | PROGRAM - copy this code and paste it inside nano (lab7b.hql)

```
USE lab_hive;  
ADD JAR ${env:HOME}/hive_lab/MaskSalaryUDF.jar;  
CREATE TEMPORARY FUNCTION mask_sal AS 'com.lab.hive.MaskSalaryUDF';  
SELECT name, mask_sal(CAST(salary AS STRING)) FROM employee;
```

Now save and exit nano: press **Ctrl+O**, then **Enter**, then **Ctrl+X**.

STEP 12 | RUN COMMANDS - run the UDF file

```
cd ~/hive_lab  
hive -f lab7b.hql
```

EXPECTED OUTPUT

Alice	XXXX-CONFIDENTIAL
Bob	XXXX-CONFIDENTIAL
Carol	XXXX-CONFIDENTIAL
Dave	XXXX-CONFIDENTIAL
Eve	XXXX-CONFIDENTIAL

Lab 8: Processing and Word Count on Apache Spark

Needs: only Python and Java (Hadoop is not needed).

STEP 1 | COMMANDS - install PySpark (version 3.5.3 works with Java 8) and make the lab folder

```
source ~/bda-env/bin/activate
pip install pyspark==3.5.3
mkdir -p ~/bda_labs/lab8
cd ~/bda_labs/lab8
```

STEP 2 | COMMAND - open the editor for spark_lab.py

```
nano spark_lab.py
```

STEP 3 | PROGRAM - copy this code and paste it inside nano (spark_lab.py)

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import explode, split, col, lower

# Initialize Spark Session
spark = SparkSession.builder \
    .appName("PySpark Execution Lab") \
    .master("local[*]") \
    .getOrCreate()
spark.sparkContext.setLogLevel("ERROR")      # hide INFO/WARN log lines

# Section A: Large dataset RDD processing
raw_rdd = spark.sparkContext.parallelize(range(1, 1000000))
even_sum = raw_rdd.filter(lambda x: x % 2 == 0).reduce(lambda a, b: a + b)
print(f"Sum of Evens in 1 Million Integers: {even_sum}")

# Section B: Spark DataFrame word count
data_df = spark.createDataFrame([
    (1, "Apache Spark handles large scale data processing efficiently."),
    (2, "Spark streaming and Spark SQL are essential components.")
], ["id", "text"])

# Tokenize and count words
words_df = data_df.select(explode(split(lower(col("text")), " ")).alias("word"))
word_counts = words_df.groupBy("word").count().orderBy(col("count").desc())

print("\nWord Count Top Results:")
word_counts.show(5)

spark.stop()
```

Now save and exit nano: press **Ctrl+O**, then **Enter**, then **Ctrl+X**.

STEP 4 | RUN COMMANDS - run the program

```
source ~/bda-env/bin/activate
python3 spark_lab.py
```

EXPECTED OUTPUT

```
Sum of Evens in 1 Million Integers: 249999500000

Word Count Top Results:
+-----+-----+
| word|count|
+-----+-----+
| spark|    3|
| ...|    1|
+-----+-----+
only showing top 5 rows
```

"spark" appears 3 times. All other words appear once, so which four words fill the other rows can change from run to run.

Lab 9: Spark Structured Streaming (Retail Dataset)

Needs: only Python and Java (Hadoop is not needed).

STEP 1 | COMMANDS - make the lab folder and create the input folder with sample JSON files

```
source ~/bda-env/bin/activate
mkdir -p ~/bda_labs/lab9
cd ~/bda_labs/lab9

mkdir -p /tmp/retail_stream_input
rm -f /tmp/retail_stream_input/*

cat > /tmp/retail_stream_input/sales1.json << 'EOF'
{"InvoiceNo":"536365","StockCode":"85123A","Quantity":10,"UnitPrice":12.5,"Country":"United Kingdom"}
{"InvoiceNo":"536366","StockCode":"71053","Quantity":4,"UnitPrice":25.0,"Country":"United Kingdom"}
{"InvoiceNo":"536367","StockCode":"84406B","Quantity":3,"UnitPrice":30.0,"Country":"Germany"}
{"InvoiceNo":"536368","StockCode":"22752","Quantity":2,"UnitPrice":15.5,"Country":"France"}
EOF

cat > /tmp/retail_stream_input/sales2.json << 'EOF'
{"InvoiceNo":"536369","StockCode":"21730","Quantity":2,"UnitPrice":20.0,"Country":"United Kingdom"}
{"InvoiceNo":"536370","StockCode":"22633","Quantity":5,"UnitPrice":10.0,"Country":"Germany"}
EOF
```

STEP 2 | COMMAND - open the editor for structured_streaming.py

```
nano structured_streaming.py
```

STEP 3 | PROGRAM - copy this code and paste it inside nano (structured_streaming.py)

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import expr
from pyspark.sql.types import DoubleType, IntegerType, StringType, StructType

spark = SparkSession.builder \
    .appName("RetailStructuredStreaming") \
    .master("local[*]") \
    .config("spark.sql.shuffle.partitions", "2") \
    .getOrCreate()
spark.sparkContext.setLogLevel("ERROR")

# Define schema for retail transactions
schema = StructType() \
    .add("InvoiceNo", StringType()) \
    .add("StockCode", StringType()) \
    .add("Quantity", IntegerType()) \
    .add("UnitPrice", DoubleType()) \
    .add("Country", StringType())

# Read streaming JSON files placed in a directory (one file per micro-batch)
streaming_df = spark.readStream \
    .schema(schema) \
    .option("maxFilesPerTrigger", 1) \
    .json("/tmp/retail_stream_input")

# Compute total spend per country (rolling aggregation)
processed_df = streaming_df \
    .withColumn("TotalSpend", expr("Quantity * UnitPrice")) \
    .groupBy("Country") \
    .sum("TotalSpend")

# Output to console sink
query = processed_df.writeStream \
    .outputMode("complete") \
    .format("console") \
    .start()

# Process the stream for 20 seconds, then stop cleanly
query.awaitTermination(20)
query.stop()
spark.stop()
```

Now save and exit nano: press Ctrl+O, then Enter, then Ctrl+X.

STEP 4 | RUN COMMANDS - run the program (it stops by itself after about 20 seconds)

```
cd ~/bda_labs/lab9
source ~/bda-env/bin/activate
python3 structured_streaming.py
```

EXPECTED OUTPUT

Batch: 0

Country	sum(TotalSpend)
United Kingdom	225.0
Germany	90.0
France	31.0

Batch: 1

Country	sum(TotalSpend)
United Kingdom	265.0
Germany	140.0
France	31.0

Batch 1 shows the rolling (cumulative) totals after the second file. Row order can differ, and if Spark reads the files in the other order the batch 0 numbers will differ.

Lab 10: Event Streaming using Apache Kafka

Objective: A Kafka producer sends events and a consumer receives them. This lab needs **four terminal windows** (open a new one with Ctrl+Alt+T, or Ctrl+Shift+T for a new tab).

STEP 1 | COMMANDS - install Kafka 3.7.0 (works with Java 8) and the Python Kafka library

```
cd ~
wget https://archive.apache.org/dist/kafka/3.7.0/kafka_2.13-3.7.0.tgz
tar -xzf kafka_2.13-3.7.0.tgz
mv kafka_2.13-3.7.0 ~/kafka

source ~/bda-env/bin/activate
pip install kafka-python-ng
mkdir -p ~/bda_labs/lab10
```

STEP 2 | COMMANDS - TERMINAL 1 - start ZooKeeper (leave it running)

```
cd ~/kafka
bin/zookeeper-server-start.sh config/zookeeper.properties
```

STEP 3 | COMMANDS - TERMINAL 2 - start the Kafka broker (leave it running)

```
cd ~/kafka
bin/kafka-server-start.sh config/server.properties
```

STEP 4 | COMMANDS - TERMINAL 3 - create the topic and check it

```
cd ~/kafka
bin/kafka-topics.sh --create --topic lab-events \
  --bootstrap-server localhost:9092 --partitions 1 --replication-factor 1
bin/kafka-topics.sh --list --bootstrap-server localhost:9092
```

The --list command must print lab-events. The backslash at the end of the first line is part of the command (or type the create command as one single line).

STEP 5 | COMMANDS - TERMINAL 3 - go to the lab folder for the consumer

```
source ~/bda-env/bin/activate
cd ~/bda_labs/lab10
```

STEP 6 | COMMAND - open the editor for consumer.py

```
nano consumer.py
```

STEP 7 | PROGRAM - copy this code and paste it inside nano (consumer.py)

```
import json
from kafka import KafkaConsumer

consumer = KafkaConsumer(
    'lab-events',
    bootstrap_servers=['localhost:9092'],
    auto_offset_reset='earliest',
    value_deserializer=lambda m: json.loads(m.decode('utf-8'))
)

print("Listening for incoming stream events...")
for message in consumer:
    event = message.value
    print(f"Received Event: {event['event_id']} | Device: {event['device']} | "
          f"Status: {event['status']}")
```

Now save and exit nano: press **Ctrl+O**, then **Enter**, then **Ctrl+X**.

STEP 8 | RUN COMMANDS - TERMINAL 3 - start the consumer first (stop it later with Ctrl+C)

```
python3 consumer.py
```

STEP 9 | COMMANDS - TERMINAL 4 - go to the lab folder for the producer

```
source ~/bda-env/bin/activate
cd ~/bda_labs/lab10
```

STEP 10 | COMMAND - open the editor for producer.py

```
nano producer.py
```

STEP 11 | PROGRAM - copy this code and paste it inside nano (producer.py)

```
import json
import time
from kafka import KafkaProducer

producer = KafkaProducer(
    bootstrap_servers=['localhost:9092'],
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

data_stream = [
    {"event_id": 1, "device": "sensor_A", "status": "NORMAL"},
    {"event_id": 2, "device": "sensor_B", "status": "WARNING"},
    {"event_id": 3, "device": "sensor_A", "status": "CRITICAL"}
]

for record in data_stream:
    producer.send('lab-events', record)
    print(f"Produced: {record}")
    time.sleep(1)

producer.flush()
producer.close()
```

Now save and exit nano: press **Ctrl+O**, then **Enter**, then **Ctrl+X**.

STEP 12 | RUN COMMANDS - TERMINAL 4 - run the producer

```
python3 producer.py
```

EXPECTED OUTPUT

```
Terminal 4 (producer):
Produced: {'event_id': 1, 'device': 'sensor_A', 'status': 'NORMAL'}
Produced: {'event_id': 2, 'device': 'sensor_B', 'status': 'WARNING'}
Produced: {'event_id': 3, 'device': 'sensor_A', 'status': 'CRITICAL'}

Terminal 3 (consumer):
Listening for incoming stream events...
Received Event: 1 | Device: sensor_A | Status: NORMAL
Received Event: 2 | Device: sensor_B | Status: WARNING
Received Event: 3 | Device: sensor_A | Status: CRITICAL
```

Quick Troubleshooting Guide

Error message	Cause	Fix
hdfs: command not found	PATH not loaded	source ~/.bashrc; check Lab 1 (set environment variables).
Connection refused ... localhost:9000	HDFS is not running	start-dfs.sh then jps; if a daemon is missing read the logs in \$HADOOP_HOME/logs.
ResourceManager / NodeManager missing in jps	YARN not configured or not started	Do the "configure YARN" step of Lab 1, then stop-all.sh and start-all.sh.
MapReduce job stays at "ACCEPTED" or container is killed	YARN has no memory / virtual memory check	Check the vmem-check line in yarn-site.xml (Lab 1), close other programs, restart with stop-all.sh and start-all.sh.
localhost: Permission denied (publickey)	Passwordless SSH not set up	Repeat the SSH step of Lab 1; make sure the ssh server is running.
JAVA_HOME is not set	Missing in hadoop-env.sh	Repeat the first line of the configure step in Lab 1.
Permission denied on an HDFS path	Wrong user or folder owner	Do NOT use sudo with hdfs. Run as the same user that started HDFS and create folders under /user/hadoop as in Lab 3.
Output directory ... already exists	MapReduce will not overwrite output	hdfs dfs -rm -r -f /wc/output
Name node is in safe mode	NameNode just started	Wait 30 seconds, or hdfs dfsadmin -safemode leave.
DataNode disappears from jps after re-formatting	clusterID mismatch	Lab machine only (deletes HDFS data): stop-dfs.sh; rm -rf ~/hadoop_data/*; mkdir -p ~/hadoop_data/namenode ~/hadoop_data/datanode; hdfs namenode -format; start-dfs.sh
externally-managed-environment (pip)	Ubuntu blocks system-wide pip	Use the virtual environment (Lab 1).
ClassNotFoundException: WeatherDriver	Driver class missing or not in jar	Create all 3 Java files of Lab 4b and rebuild the jar.
error: class X is public, should be declared in a file named X.java	Wrong file name in nano	File name must match the class name exactly.
Hive: NoSuchMethodError ... guava	Guava jar clash	Do the Guava fix in Lab 7a.
Spark: UnsupportedClassVersionError or Java version error	PySpark 4.x with Java 8	pip install pyspark==3.5.3
No module named 'kafka.vendor.six.moves'	Old kafka-python on Python 3.12	pip uninstall kafka-python; pip install kafka-python-ng
NoBrokersAvailable	Kafka broker not running	Start ZooKeeper first, then the broker (Lab 10, terminals 1 and 2).

Daily start-up order: 1) open the terminal and run `sudo su - hadoop`; 2) `source ~/bda-env/bin/activate` (Python labs); 3) `start-all.sh` and check `jps` (Labs 3, 4, 5, 7); 4) do the lab. When finished: `stop-all.sh`.